Capitolo 4

Testing, JUnit e verifica della qualità

4.1 Casi di test, oracle e debugging

Il **testing** consiste nell'eseguire il codice in determinate condizioni e nel valutare se il risultato ottenuto è corretto. Un **test case** (caso di test) contiene almeno dati di ingresso, stato iniziale o contesto, risultato atteso e risultato osservato durante l'esecuzione.

Il risultato atteso è l'**oracle**, cioè il riferimento con cui confrontare il comportamento effettivo. Per un sistema nuovo l'oracle deriva dai requisiti; durante il refactoring può essere il comportamento della versione precedente.

Un test superato indica che, nelle condizioni controllate, il comportamento osservato coincide con quello atteso; un test fallito segnala una differenza che occorre esaminare. Dal punto di vista della ricerca dei difetti, un test è utile proprio quando ne rende visibile uno.

Il **debugging** consiste nel localizzare e correggere la causa della discrepanza, che può trovarsi nel programma, ma anche nell'oracle o nella preparazione del test.

I test documentano inoltre il comportamento richiesto e proteggono dalle modifiche indesiderate; una suite finita non dimostra normalmente l'assenza di tutti i difetti.

Procedere per ipotesi nel debugging

Chi esegue il debugging formula un'ipotesi sulla causa e prepara una prova capace di confermarla o smentirla. La prova può essere un'esecuzione oppure un'osservazione; il suo risultato permette di circoscrivere il problema e scegliere il controllo successivo.

Cambiare più parti senza un'ipotesi rende difficile capire quale modifica abbia prodotto il risultato. Occorre controllare anche il test: il difetto può trovarsi nella specifica, nella logica o nella fixture, cioè nei dati e nello stato preparati per la prova.

4.2 Verifica, validazione e livelli di test

La **verifica** risponde alla domanda "stiamo costruendo il sistema nel modo corretto?": controlla implementazione, qualità del codice e rispetto della progettazione. La **validazione** risponde alla domanda "stiamo costruendo il sistema corretto?": il riferimento sono i bisogni e le richieste dell'utente.

Questa distinzione riguarda lo scopo del controllo. White-box e black-box indicano invece due prospettive per progettare i test: la prima esamina la struttura interna, la seconda il comportamento rispetto alle specifiche. Non sono definizioni alternative di verifica e validazione.

I difetti possono nascere nella specifica, per esempio da requisiti ambigui, contraddittori o imprecisi; nella progettazione, per esempio da algoritmi, strutture dati o interfacce fra moduli non corretti; nel codice, cioè nell'implementazione; oppure negli stessi casi e piani di test. I test dovrebbero essere affiancati alle *verifiche statiche*, cioè controlli sul codice svolti senza eseguirlo. Possono essere organizzati per livello:

test unitari isolano un singolo componente, quale metodo, oggetto, funzione o piccolo raggruppamento di essi;

test di integrazione esercitano la collaborazione fra componenti e le rispettive interfacce, eventualmente usando database, rete o file system reali;

test di sistema verificano il sistema complessivo, comprese funzionalità e proprietà emergenti dalle interazioni;

alpha test sono condotti dagli sviluppatori usando dati reali del cliente per controllare che le richieste siano soddisfatte;

beta test sono condotti dai clienti su un prodotto quasi completo.

L'uso di dati reali del cliente negli alpha test richiede modalità di accesso e riservatezza concordate. Un *NDA* (accordo di non divulgazione) può disciplinare questo accesso: autorizzare i dati per il test non implica autorizzarne altri usi.

Una *versione alfa* è una versione integrata che può avere difetti e parti mancanti; una *versione beta* include tutte le feature ma può ancora avere difetti; la *versione gold* è il prodotto finale pronto alla commercializzazione.

Un *test di regressione* riusa una suite dopo una modifica, una correzione di bug o una nuova funzionalità per accertare che ciò che funzionava continui a funzionare. Una *test suite* è un insieme di test case eseguiti insieme.

Per escludere la presenza di difetti mediante la sola esecuzione, occorrerebbe controllare tutte le condizioni rilevanti con un oracle corretto. Il numero di prove rende questo obiettivo normalmente impraticabile.

Consideriamo una funzione con due parametri di tipo `int`. Assumendo 2^{32} valori possibili per ciascun parametro, dobbiamo combinare ogni valore del primo con ogni valore del secondo: i casi sono $2^{32} \cdot 2^{32} = 2^{64} \simeq 1{,}8 \cdot 10^{19}$.

Se ciascun caso richiedesse $1\,\text{ns} = 10^{-9}\,\text{s}$, il tempo totale sarebbe circa $1{,}8 \cdot 10^{10}\,\text{s}$. Convertendo il tempo in anni otteniamo circa 600 anni, assumendo $1\,\text{anno} \simeq 3 \cdot 10^{7}\,\text{s}$. Anche con il tempo ipotetico molto piccolo scelto per una prova, l'esplorazione esaustiva richiederebbe quindi un tempo enorme.

Si selezionano quindi casi efficaci: quelli tipici dimostrano le capacità richieste, quelli sulle funzionalità esistenti proteggono dalle regressioni e quelli limite esplorano condizioni nelle quali sono frequenti i difetti. L'ordine delle priorità dipende anche dalla criticità delle funzionalità.

Distribuzione e priorità dei test

Una suite utile comprende molti test unitari rapidi, eseguiti frequentemente, e un numero più contenuto di test d'integrazione su risorse reali, spesso più lenti. La distribuzione dei test dipende dai rischi e dalle dipendenze, non da una proporzione fissa. Come ordine pratico iniziale si può proteggere prima il comportamento già disponibile rispetto alle nuove estensioni e controllare prima i casi tipici rispetto ai limiti; criticità elevate o difetti noti possono motivare priorità diverse.

Esempi aiutano a distinguere le origini dei difetti: un algoritmo che permette una divisione per zero, un campo necessario assente o un parametro di interfaccia di tipo incoerente possono rivelare errori di progetto. Overflow, conversione di tipo errata, precedenza degli operatori, variabili non inizializzate o riassegnate prima dell'uso possono invece derivare dall'implementazione. Il confine non è sempre netto: occorre ricostruire quale decisione ha introdotto il problema. Anche piani e casi di test possono essere difettosi.

4.3 Progettazione black-box e white-box

Un test *black-box* progetta i casi a partire dalle specifiche e dai requisiti, senza usare la struttura interna o l'algoritmo. Chi lo scrive può quindi ragionare sugli scenari normali, particolari e negativi senza essere condizionato dall'implementazione.

Un test *white-box*, detto anche *glass-box* o strutturale, richiede invece conoscenza del codice sorgente e sceglie i casi osservando rami, istruzioni e percorsi di controllo.

Nel black-box il dominio degli input può essere partizionato in **classi di equivalenza**: ciascuna è un insieme di dati per i quali si prevede un comportamento uniforme. Si sceglie almeno un rappresentante per classe e, in particolare, valori ai suoi estremi, dove i difetti sono frequenti. Ciò riduce molti possibili input a un insieme gestibile di casi rappresentativi.

Occorre includere sia dati validi sia dati non validi; un *fault model*, cioè un'ipotesi sul tipo di difetto che si vuole cercare, può riguardare un intero intervallo che l'implementazione elabora in modo scorretto.

Nel white-box i *test di percorso* cercano di eseguire almeno una volta ogni percorso indipendente del programma. Nel grafo di controllo i nodi rappresentano punti del programma, come condizioni o blocchi di istruzioni, e gli archi i possibili flussi; la complessità ciclomatica misura il numero di percorsi indipendenti da considerare. Coprire i percorsi indipendenti non implica coprire ogni combinazione possibile di condizioni.

Un *test sotto stress* esegue il sistema oltre il carico massimo previsto e osserva che il fallimento, se avviene, non sia catastrofico; è rilevante soprattutto per sistemi distribuiti.

4.3.1 Intervallo ammesso, confini e stress

Supponiamo che il contratto ammetta interi da 4 a 20. Dividiamo gli input nelle partizioni $x < 4$, $4 \leq x \leq 20$ e $x > 20$. I casi 3, 4, 12, 20 e 21 esercitano rispettivamente una classe non valida, il confine inferiore valido, un valore interno, il confine superiore valido e l'altra classe non valida.

La scelta di un rappresentante presume un comportamento omogeneo nella partizione. Serve a cercare difetti con un numero gestibile di prove, ma non dimostra che l'implementazione tratti

tutti i valori in modo equivalente. I confini meritano prove specifiche perché un errore come `x<20` al posto di `x<=20` cambia proprio l'ammissione del valore al limite.

I difetti possono addensarsi in componenti già problematici: complessità o precedenti fallimenti suggeriscono altri casi mirati, senza escludere le parti mai fallite.

Nello stress si cercano conseguenze come perdita di dati o indisponibilità del servizio. Nei sistemi distribuiti conta anche il degrado della rete. I risultati possono richiedere di completare le specifiche sul comportamento oltre il carico previsto e sulle modalità di recupero.

4.4 Osservabilità, contratti e indipendenza dei test

Per testare un oggetto, lo stato che interessa verificare deve essere *osservabile*, cioè leggibile mediante metodi appropriati, per esempio getter o query. Una *query* restituisce un'informazione senza modificare lo stato. Se non è possibile osservare l'effetto rilevante, il test non può confrontarlo con l'oracle. Un metodo difficile da testare può segnalare che svolge troppi compiti e non rispetta il Principio di Singola Responsabilità (SRP).

Una **precondizione**, nel Design by Contract, è una condizione che deve essere vera prima di eseguire correttamente un metodo. Per esempio, per accedere all'elemento di indice i di un array o di una lista di dimensione n, deve valere $i \geq 0$ e $n > i$: i è l'indice richiesto e n il numero di elementi. I test devono esplicitare anche i casi negativi, quali lista vuota, indice fuori intervallo e limiti minimo e massimo.

I controlli iniziali, o *guard clause*, permettono di trattare il caso negativo prima del percorso principale; l'esito previsto, compresa un'eventuale eccezione, deve essere stabilito dal contratto.

Un buon test rispetta i principi *FIRST*:

Fast è veloce, così la suite può essere eseguita frequentemente;

Isolated/Independent è isolato e indipendente: non dipende dall'ordine di esecuzione né dallo stato lasciato da un altro test;

Repeatable è ripetibile: a parità di input dà lo stesso esito su macchine e momenti diversi;

Self-validating si autovaluta e comunica automaticamente passaggio o fallimento, con un messaggio chiaro;

Timely è scritto nel momento opportuno, idealmente prima del codice quando si usa TDD.

I test devono restare semplici: se un test complesso fallisce, può essere difficile capire se il difetto sia nel test o nel sistema. Come criterio pratico di organizzazione, si prepara almeno una classe di test per ciascuna classe dell'applicazione e almeno un test per ciascun metodo significativo.

Precisazione

Rendere osservabile un effetto non significa rendere accessibili tutti i campi interni senza un criterio. È preferibile una query pubblica coerente con il dominio o la verifica di un risultato già previsto dal contratto. Se l'unico modo di testare un metodo è leggere molte variabili intermedie, può essere utile separarne le responsabilità.

Le eccezioni vanno gestite al livello che ha le informazioni necessarie per recuperare dall'errore, oppure propagate o tradotte secondo il contratto. Un `catch` vuoto può nascondere un difetto;

allo stesso modo, restituire `null` nel caso negativo richiede una decisione esplicita e coerente con l'uso del metodo.

Quando una condizione negativa è prevista e ordinaria, è spesso più chiaro controllarla con una guard clause anziché usare un'eccezione per gestire il normale flusso del programma. Una libreria va inoltre incapsulata se i suoi errori espongono dettagli estranei al dominio: il livello applicativo può tradurli o gestirli. Ciò non introduce un divieto generale di propagare eccezioni Java; il contratto deve chiarire questa scelta, che va affidata al livello capace di decidere come recuperare dall'errore.

4.5 TDD, BDD e struttura del caso di test

Il **Test-Driven Development** (TDD) è una pratica di sviluppo in cui si scrive prima il test e poi il codice minimo necessario a soddisfare il requisito. Il suo ciclo è:

1. **Red**: si scrive un test non ancora superato; nello scegliere nomi di classi e metodi si compie già progettazione;
2. **Green**: si implementa il minimo codice che fa passare il test;
3. **Refactor**: si migliora la struttura senza cambiare il comportamento osservabile, quindi si riesegue la suite.

Il principio YAGNI limita l'implementazione alle funzionalità richieste, evitando estensioni speculative. I test guidano il passo corrente, ma averli tutti superati non dimostra che ogni requisito sia stato formulato o verificato correttamente.

La struttura *AAA* organizza il singolo test: *Arrange* prepara lo stato, *Act* chiama l'operazione e *Assert* confronta il risultato osservato con quello atteso.

Il *Behaviour-Driven Development* (BDD) esprime il comportamento dal punto di vista dell'utente mediante *Given–When–Then*, abbreviato GWT: dato un contesto, quando avviene un'azione, allora si produce un effetto atteso. Questa forma rende il test leggibile anche come documentazione. Le tre parti corrispondono ad Arrange–Act–Assert: Given prepara il contesto, When esegue l'azione e Then esprime il risultato atteso.

BDD e TDD sono complementari, ma descrivono aspetti diversi. Il ciclo TDD riguarda soprattutto il processo con cui si sviluppa il codice; la descrizione BDD rende esplicito il comportamento. Si può usare quest'ultima anche senza scrivere il test prima del codice: in quel caso non si sta, per questo solo fatto, adottando il ciclo TDD.

4.5.1 Dallo scheletro compilabile al test Red

Il TDD richiede di distinguere l'errore di compilazione dal fallimento di un'asserzione. Se il test nomina una classe o un metodo ancora inesistente, prima si prepara lo scheletro minimo che consente di compilare: costruttore, firme e risultati provvisori. Si esegue quindi il test e si verifica che fallisca per il comportamento mancante, poi si implementa la funzionalità e si raggiunge Green. Uno scheletro che ritorna sempre zero non è ancora una soluzione: serve a permettere di osservare il fallimento atteso.

4.6 Copertura, percorsi e criteri di arresto

Nel grafo di flusso usato dai test di percorso la complessità ciclomatica è:

$$cc = A - N + 2. \tag{4.1}$$

Qui cc è la complessità ciclomatica, A il numero di archi e N il numero di nodi di un grafo di controllo connesso.

Il valore indica quanti percorsi contiene un insieme di percorsi indipendenti: orienta la progettazione dei test strutturali, senza rendere esaustive tutte le combinazioni di esecuzione.

Una sequenza priva di diramazioni ha $cc = 1$. Una condizione, rappresentata da un nodo decisionale e uno di uscita con due archi alternativi, ha $cc = 2 - 2 + 2 = 2$: occorrono casi che esercitino entrambi gli esiti.

Il ritorno alla condizione in un ciclo

Nel grafo di un `while` distinguiamo tre nodi: la condizione, il corpo del ciclo e il punto di uscita. Quando la condizione è vera, un arco conduce al corpo; al termine del corpo un secondo arco torna alla condizione. Quando è falsa, il terzo arco conduce all'uscita. Con tre archi e tre nodi si ottiene quindi $cc = 3 - 3 + 2 = 2$.

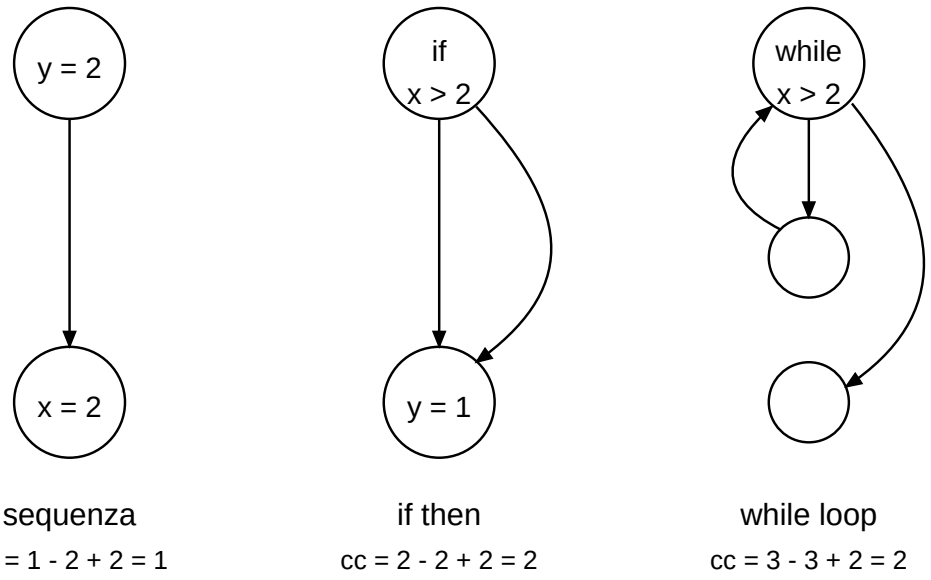


Figura 4.1: Confronto fra sequenza, condizione e ciclo: il ritorno del while alla condizione aggiunge un percorso indipendente.

Per seguire i due percorsi consideriamo un corpo che decrementa `x`:

```
while (x > 2) {
    x--;
}
```

Con `x=2` il corpo non viene eseguito: si passa dalla condizione all'uscita. Con `x=3` si entra nel corpo una volta, si decrementa il valore, si torna alla condizione e infine si esce. Le due prove

esercitano i due percorsi indipendenti di questo esempio. Un valore iniziale maggiore provoca più ripetizioni, ma non aggiunge una decisione al grafo: coprire questi percorsi non equivale a verificare ogni possibile numero di iterazioni o ogni valore elaborato dal corpo.

La **code coverage** misura quale parte del codice viene esercitata dalla suite secondo un criterio dichiarato. Per la copertura delle linee si usa:

$$C = \frac{L_{\text{eseguite}}}{L_{\text{totali}}} \cdot 100\%. \tag{4.2}$$

Qui C è la copertura, L_{eseguite} il numero di linee raggiunte almeno una volta e L_{totali} il numero totale di linee considerate. Per esempio, $C = 70\%$ indica che è stato eseguito il 70% delle linee. Anche il 100% lascia aperta la possibilità di input, combinazioni o requisiti non verificati.

Soglie del 90–95% possono orientare il lavoro, ma vanno motivate in base al sistema e non garantiscono correttezza; le parti critiche possono richiedere controlli più rigorosi. Il tracciamento della copertura può essere integrato nell'ambiente di sviluppo.

Criteri diversi misurano aspetti diversi:

- la copertura di funzioni conta le funzioni eseguite;
- quella di costrutti le istruzioni;
- quella di archi i flussi attraversati nel grafo di controllo.

La copertura di rami richiede di esercitare le alternative, per esempio il ramo `if` e il ramo `else`; quella di condizioni richiede che ogni sottoespressione booleana assuma almeno una volta valore vero e falso.

La copertura di decisioni riguarda gli esiti delle espressioni che determinano il flusso; l'esecuzione dei punti di ingresso e uscita è un controllo ulteriore, da esplicitare quando si combinano più criteri.

Per `if (x && y) a=1; else a=0;`, i casi $(x, y) = (vero, vero)$ e $(falso, falso)$ raggiungono entrambi gli esiti della decisione. In Java, nel secondo caso `y` non viene valutato per il corto circuito: per osservare anche `y=false` serve, per esempio, $(vero, falso)$. Occorre quindi distinguere valori assegnati alle variabili e sottoespressioni realmente valutate.

La copertura delle decisioni controlla i due esiti della decisione composta; combinandola con la copertura delle funzioni e dei punti di ingresso e uscita si ottiene un criterio strutturale più ampio. Quando il termine è usato per tale criterio composto, vanno dichiarate esplicitamente tutte queste componenti; non implica la verifica di ogni possibile combinazione di input.

Unità di misura e soglie di copertura

Il rapporto di copertura dipende dall'unità che si sceglie di misurare: linee eseguite su linee totali, rami eseguiti su rami totali, e così via. Una soglia del 60% può illustrare un obiettivo iniziale, come quelle del 90–95% illustrano obiettivi più elevati: nessuna delle percentuali costituisce una garanzia.

Una build notturna, o *nightly build*, può compilare e rieseguire una suite ampia come controllo di regressione, affiancando i test rapidi eseguiti durante le modifiche.

Il **bug trend** misura la frequenza con cui si scoprono difetti. Insieme alla copertura desiderata, orienta la decisione di arrestare il testing.

Una frequenza che tende a zero ammette letture diverse: possono non emergere nuovi difetti, oppure può essere diventato eccessivamente costoso trovarne altri. Il dato va quindi interpretato e non costituisce, da solo, una prova di correttezza.

Nell'IDE, il tracciamento della copertura usa il verde per linee eseguite almeno una volta e il rosso per linee non raggiunte. Questo colore riguarda l'esecuzione, mentre Red e Green del TDD riguardano l'esito dei test: una linea eseguita può contenere un errore che l'oracle non rileva. I test dell'usabilità dell'interfaccia richiedono inoltre valutazioni che non si riducono facilmente a unit test automatizzati.

4.7 JUnit: annotazioni, asserzioni e fixture

JUnit è il framework per scrivere ed eseguire test di applicazioni Java. Fornisce un motore di esecuzione, o *test runner*, e API per definire test e asserzioni.

Un'*annotazione* Java è un metadato associato a elementi del codice, come classi, metodi e campi, che fornisce informazioni aggiuntive agli strumenti. In JUnit:

`@Test` identifica un metodo di test;

`@BeforeEach` identifica un metodo eseguito prima di ogni test, utile per ottenere uno stato fresco;

`@BeforeAll` identifica un metodo eseguito una sola volta prima di tutti i test della classe; nella forma usata qui il metodo è `static`;

`@ParameterizedTest` indica un test eseguito più volte con input differenti; `@CsvSource` può fornire tali input in forma tabellare CSV.

Le *asserzioni* della classe `Assertions` effettuano il confronto automatico con l'oracle. `assertEquals`, applicata agli argomenti `expected` e `actual`, verifica l'uguaglianza fra valore atteso e valore effettivo; `assertTrue` verifica una condizione booleana vera; `assertNull` verifica un riferimento `null`.

Per risultati in virgola mobile occorre una tolleranza: se r è il risultato, a il valore atteso e $\varepsilon = 0{,}01$ la tolleranza, una verifica esplicita della distanza fra i valori è

$$|r - a| < \varepsilon. \tag{4.3}$$

4.7.1 Catalogo: test di somma e massimo

Un `Catalogo` conserva prodotti inseriti come stringhe CSV, cioè campi separati da virgole: nome, prezzo, responsabile e data. `svuota()` elimina i prodotti, `inserisci(...)` ne aggiunge uno, `calcolaSomma()` aggiorna il totale letto da `getSomma()`, mentre `calcolaMassimo()` aggiorna il massimo letto da `getMassimo()`.

La preparazione condivisa del test, detta **fixture**, usa quattro prezzi: 12,00, 10,50, 2,00 e 21,50. Il totale atteso è 46,00 e il massimo 21,50. `@BeforeAll` prepara i dati una sola volta; i metodi `@Test` controllano le due operazioni.

```
public class AppTest {
    private static final Catalogo catalog = new Catalogo();
```

```
    @BeforeAll
    public static void init() {
        catalog.svuota();
        catalog.inserisci("album,12.00,Rick,2025-01-15");
        catalog.inserisci("libro,10.50,Rick,2025-01-15");
        catalog.inserisci("matite,2.00,Rick,2025-01-15");
        catalog.inserisci("zaino,21.50,Rick,2025-01-15");
    }

    @Test
    public void sommaValori() {
        catalog.calcolaSomma();
        assertEquals(46.00f, catalog.getSomma(), 0.01f);
    }

    @Test
    public void massimoValori() {
        catalog.calcolaMassimo();
        assertEquals(21.50f, catalog.getMassimo(), 0.01f);
    }
}
```

In ciascun test il metodo di calcolo esegue l'azione; il getter rende osservabile il risultato e l'asserzione lo confronta con l'oracle. Il test si autovaluta: non occorre leggere manualmente i messaggi `OK`/`FAILED`.

La fixture è però condivisa e inizializzata una sola volta. L'esempio mostra l'uso dell'annotazione; per rispettare l'indipendenza FIRST occorre verificare che un test non modifichi lo stato in modo da influenzare l'altro. Quando questa condizione non è garantita, è preferibile preparare uno stato nuovo con `@BeforeEach`.

Esempio: i rami del parser CSV

Il metodo di inserimento CSV offre casi concreti da osservare nella copertura:

```
String[] campi = riga.split(",");
if (campi.length != 4) {
    System.err.println("Riga malformata: " + riga);
    return;
}
if (campi[1].trim().equals("valore")) return; // intestazione
String nome = campi[0].trim();
float prezzo = Float.parseFloat(campi[1].trim());
```

Dopo i controlli il metodo costruisce il prodotto con gli altri due campi; un errore di conversione richiede un esito previsto dal contratto.

Se la suite usa solo righe corrette, il ramo della riga malformata può restare rosso anche quando la lettura del file e il ciclo sono verdi. Vanno aggiunte righe con numero di campi diverso da quattro, intestazione e prezzo non numerico, controllando l'effetto atteso. Il colore di una linea non dimostra che il parser riconosca tutti i CSV possibili: lo `split` semplice non gestisce per sé virgolette o virgole interne ai campi.

La fixture a due prodotti può essere inizializzata una volta con un campo `static` e `@BeforeAll`, ottenendo somma 33,50 e massimo 21,50. La variante seguente con `@BeforeEach` cambia intenzionalmente il ciclo di preparazione: ogni test riceve un catalogo nuovo invece di condividere la medesima istanza.

Il test parametrizzato della somma rende visibili i quattro casi nella classe CalcTest, definita insieme al metodo da controllare. Ogni riga CSV fornisce due operandi e il risultato atteso.

4.7.2 Variante con due prodotti e preparazione indipendente

Una fixture alternativa contiene soltanto album da 12,00 e zaino da 21,50: il totale atteso diventa 33,50. Preparare un nuovo catalogo con `@BeforeEach` rende esplicita l'indipendenza dei test. Valori attesi e dati iniziali devono sempre descrivere lo stesso scenario:

```
private Catalogo catalog;

@BeforeEach
void preparaCatalogo() {
    catalog = new Catalogo();
    catalog.inserisci("album,12.00,Rick,2025-01-15");
    catalog.inserisci("zaino,21.50,Rick,2025-01-15");
}

@Test
void sommaDueProdotti() {
    catalog.calcolaSomma();
    assertEquals(33.50f, catalog.getSomma(), 0.01f);
}
```

Le annotazioni di base appartengono a `org.junit.jupiter.api`. Per usare i frammenti si importano i tipi e le asserzioni necessari:

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;
```

La tolleranza rende esplicita la politica di confronto dei floating-point. Lista vuota, un solo elemento, valori ai limiti e input non validi richiedono casi ulteriori, con un comportamento atteso definito dal contratto.

4.7.3 Test eseguibili di Pagamenti e lettura da file

Il caso `Pagamenti` permette di distinguere un controllo manuale dell'output da un test che si autovaluta. La fixture inserisce 321,01, 531,7 e 1234,5: somma attesa 2087,21, massimo 1234,5. La seguente classe usa la variante con query e si può eseguire senza un framework:

```
import java.io.IOException;
```

```
class TestPagamenti {
    private final Pagamenti pgm = new Pagamenti();
    private void initLista() {
        pgm.svuota();
        pgm.inserisci(321.01f);
        pgm.inserisci(531.7f);
        pgm.inserisci(1234.5f);
    }
    private void verifica(boolean ok, String nome) {
        System.out.println((ok ? "OK " : "FAILED ") + nome);
    }
    void testSommaValori() {
        initLista();
        verifica(Math.abs(pgm.calcolaSomma() - 2087.21f) < 0.01f,
                "test somma val");
    }
    void testMaxValore() {
        initLista();
        verifica(Math.abs(pgm.calcolaMassimo() - 1234.5f) < 0.01f,
                "test massimo val");
    }
    void testVuoto() {
        pgm.svuota();
        verifica(pgm.calcolaSomma() == 0, "somma vuota");
        verifica(pgm.calcolaMassimo() == 0, "massimo vuoto");
    }
    void testLeggiFile() {
        try {
            pgm.leggiFile("csv", "Importi.csv");
            verifica(true, "test leggi file");
        } catch (IOException e) {
            verifica(false, "test leggi file");
        }
    }
    public static void main(String[] args) {
        TestPagamenti t = new TestPagamenti();
        t.testLeggiFile();
        t.testSommaValori();
        t.testMaxValore();
        t.testVuoto();
    }
}
```

Con un file leggibile e composto da numeri validi le prime tre righe iniziano con `OK`; se il file non è trovato, la prima diventa `FAILED test leggi file`, mentre somma e massimo passano perché ricreano la propria fixture.

Il test di lettura controlla soltanto l'assenza di `IOException`: per verificare il contenuto occorre confrontare anche i dati o gli aggregati attesi. Una riga non numerica richiede un caso distinto, perché non è un errore di I/O. Il confronto esatto con un letterale decimale non è appropriato per questi aggregati in virgola mobile: la tolleranza va applicata sia alla somma sia al massimo.

Nella variante con risultati memorizzati la stessa fixture si inserisce come testo, poi si eseguono `converti`, `calcolaSomma` e `calcolaMassimo`; le verifiche leggono `getSomma` e `getMassimo`. I valori attesi rimangono identici se rappresentazione dei duplicati, dominio e politica sugli

errori coincidono. Per entrambi i contratti si aggiungono casi con un duplicato, un solo valore, letture successive e lista svuotata.

Un comportamento BDD dello stesso caso è: *Given* una raccolta contenente i tre importi distinti; *When* ne richiedo la somma; *Then* ottengo 2087,21 entro la tolleranza dichiarata. L'oracle viene così espresso prima dell'implementazione scelta.

4.7.4 Un metodo con input intero: scheletro, test e soluzione

Per provare un metodo che riceve un intero si può specificare prima l'addizione: `add(2,3)` deve restituire 5. Lo scheletro `int add(int a,int b) { return 0; }` compila ma fallisce il test; sostituire il corpo con `return a+b;` realizza il comportamento per il dominio considerato. Un test parametrizzato completo rende espliciti i quattro casi:

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.assertEquals;

class Calc {
    int add(int a, int b) { return a + b; }
}
class CalcTest {
    private final Calc calculator = new Calc();
    @ParameterizedTest
    @CsvSource({"1,1,2", "2,3,5", "10,5,15", "-1,1,0"})
    void testAdd(int a, int b, int expected) {
        assertEquals(expected, calculator.add(a, b));
    }
}
```

Questi casi non specificano come trattare l'overflow degli interi: per estendere il requisito oltre l'aritmetica `int` occorrerebbe un contratto ulteriore.

4.8 Maven: progetto, dipendenze e ciclo di build

Apache Maven automatizza il ciclo di build, cioè le attività che portano dai sorgenti al prodotto distribuibile: gestisce dipendenze, test, packaging e rilascio.

Nella struttura convenzionale, il codice applicativo si trova in `src/main/java` e i test in `src/test/java`. Il file `pom.xml`, il Project Object Model (POM), descrive il progetto elencando nome, versione, dipendenze, plugin e configurazioni. Lo scope limita l'ambito di una dipendenza: con scope `test` la libreria è disponibile solo per i test.

Le fasi `mvn compile`, `mvn test`, `mvn package` e `mvn verify` eseguono, nell'ordine, compilazione, test unitari, produzione del file distribuibile `.jar` e controlli di qualità. Quando si richiede una fase, Maven esegue anche quelle precedenti: per produrre il pacchetto passa quindi anche attraverso compilazione e test.

Una dipendenza è identificata da coordinate e versione; Maven scarica anche le dipendenze transitive. Per esempio, una configurazione che usa JUnit 6 può fissare esplicitamente la versione `6.0.3`, mantenendo coerenti librerie, JDK e plugin. Test di integrazione e controlli

aggiuntivi richiedono una configurazione adeguata: la presenza della fase `verify` non attiva automaticamente qualsiasi verifica.

```
progetto/
  pom.xml
  src/main/java/com/example/   # codice applicativo
  src/test/java/com/example/   # classi di test
```

4.8.1 Creare il progetto e fissare la configurazione

Si può creare la struttura con l'IDE oppure con `mvn archetype:generate`, scegliendo un archetipo Java e le coordinate del progetto. Per aggiungere una libreria si cercano in Maven Central gruppo, artefatto e versione; si copia il blocco `dependency` nella sezione `dependencies` del POM e si assegna lo scope appropriato. Le dipendenze transitive sono le librerie richieste a loro volta dalla dipendenza dichiarata: non occorre ricopiare manualmente ogni archivio.

Un POM completo per gli esempi con Java 25 e JUnit 6.0.3 è:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>prj-junit6</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>6.0.3</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>6.0.3</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <version>6.0.3</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
```

```
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.14.1</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
      </plugin>
    </plugins>
  </build>
</project>
```

`groupId`, `artifactId` e `version` identificano il progetto; `modelVersion` identifica il modello del POM. `release` seleziona livello del linguaggio, bytecode e API della piattaforma obiettivo: per questo non occorre ripetere anche `source` e `target` con lo stesso valore.

Il JDK usato deve supportare la release selezionata. L'API permette di scrivere i test, l'engine li esegue e `params` abilita le sorgenti dei test parametrizzati. Il plugin compiler applica la configurazione Java; Surefire avvia i test nel ciclo Maven. Fissare le versioni permette di riprodurre la configurazione; questo non significa che siano le uniche versioni possibili.

Un JAR, *Java Archive*, è un archivio compresso di classi compilate e risorse, eventualmente con metadati di avvio. `mvn package` lo produce nella cartella `target`; non implica che ogni JAR sia avviabile direttamente. La verifica della suite si ottiene con `mvn test`, mentre `verify` comprende le fasi precedenti e le ulteriori verifiche effettivamente configurate.

4.9 Procedimento per progettare ed eseguire un test

Per progettare ed eseguire un test si può seguire questa sequenza:

1. identificare il requisito, il componente e il comportamento da controllare;
2. stabilire oracle, precondizioni, stato iniziale e input validi, non validi e di confine; per un black-box scegliere rappresentanti delle classi di equivalenza;
3. rendere osservabile il risultato mediante l'interfaccia pubblica o query appropriate;
4. organizzare il caso in Arrange–Act–Assert, predisponendo uno stato indipendente dagli altri test;
5. formulare l'asserzione; per valori floating-point applicare una tolleranza anziché uguaglianza esatta;
6. eseguire la suite con il runner, interpretare un fallimento come segnale da investigare e correggere con debugging;
7. dopo ogni modifica rieseguire la suite come regressione e osservare la copertura senza confonderla con la correttezza.